Table of Contents

1.Abstract: 2
2.Introduction: 2
3.Methodology: 3
4.Implementation:..... 5
5.Results & Analysis:……6
6.Discussion: 8
7.Conclusion:..... 9

PROJECT REPORT

GUI BASED CAR RACING GAME USING JAVA

1. **Abstract**

This project focuses on developing an interactive, 2D arcade-style application called the Car Racing Game using the Java programming language. The main objective addressed in this project is creating a real-time GUI application that allows users to control a vehicle, avoid obstacles, manage state transitions, and experience embedded audio feedback. The system uses custom vector drawing via Java Swing and AWT to render a dynamic driving track, player car, and randomized enemy vehicles. The methods implemented include continuous timer-based rendering (~60 FPS), bounding-box collision detection (Rectangle.intersects), keyboard event listening (KeyListener), finite state management, and real-time audio stream handling using the Java Sound API (javax.sound.sampled). The results show that the application reliably updates frame physics, tracks user performance metrics (score and survival time), and manages game states seamlessly. This project demonstrates the practical application of object-oriented design and real-time graphics handling in Java.

2. **Introduction**

-Background

Java is an object-oriented programming language widely recognized for its platform independence, robust memory management, and rich graphical library support. Developing 2D interactive applications provides an effective way to master event-driven programming, multi-threading, and graphical user interface (GUI) development. Swing and AWT frameworks allow developers to manipulate hardware-accelerated 2D graphics, process real-time user keyboard inputs, and manage dynamic animation loops using built-in timers.

-Problem Statement

Understanding event-driven GUI architecture, real-time animation loops, and spatial collision physics can be challenging when approached solely through theoretical concepts. Standard non-interactive learning often misses the complexity of frame management, visual state tracking, and audio synchronization. Therefore, constructing an interactive 2D racing game addresses these educational challenges by applying object-oriented design to solve real-time rendering and input-handling problems.

-Objectives

The objectives of this project are:

- To design and implement an interactive 2D Car Racing Game using Java Swing and AWT.

- To implement a frame-rate driven game loop using javax.swing.Timer executing at ~60 FPS.
- To apply AWT bounding-box collision detection (java.awt.Rectangle) for collision handling.
- To build a Finite State Machine managing Welcome, Active Gameplay, and Game Over states.
- To integrate multi-channel audio handling (.wav audio streams) using javax.sound.sampled.
- To provide keyboard controls (KeyListener) for vehicle movement, game initialization, and system restarts.

-Scope

The scope of this project includes a complete 2D desktop application built with Java Swing and AWT (JPanel, JFrame, Graphics2D). The application features dynamic road animation, dynamic obstacle generation, heads-up display (HUD) statistics, multi-state menu flows, and responsive sound effects. The project operates in a local desktop environment and does not include network multiplayer functionality or database persistence.

3. **Methodology**

-Approach

The project follows an event-driven, object-oriented approach. A javax.swing.Timer triggers continuous action events every 16 milliseconds (~60 FPS). During each timer tick, the system updates entity coordinates, updates background animation offsets, increments survival time counters, checks bounding-box intersections between vehicles, and requests GUI repaints.

-Algorithm

Algorithm for 2D Car Racing Game State & Loop Logic:

1. Start the application and initialize window dimensions (600 X 700 pixels).
2. Set default parameters: player car position (260, 500), enemy car position (260, -120), score = 0, survival time = 0, and set state flags (isWelcomeScreen = true, isGameOver = false).
3. Start background audio loop (welcome.wav) and trigger the 60 FPS timer.
4. Intercept user keyboard inputs (KeyListener):

    - If on Welcome Screen and **ENTER** is pressed: transition to gameplay (isWelcomeScreen = false), stop welcome audio, and start engine sound (run.wav).
    - If during Gameplay and **LEFT** / **RIGHT** keys are pressed: update player X-coordinate within road boundaries (150 < playerX < 390).
    - If on Game Over Screen and **R** is pressed: reset positions, score, timer, and re-engage gameplay.

5. Update state variables during each timer event:

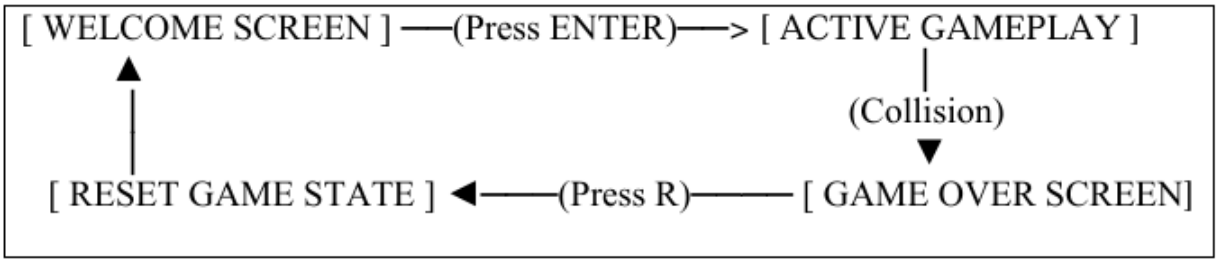
    - Increment vertical road offset (roadOffsetY += 5) to simulate forward motion.
    - Advance enemy vehicle downward (enemyY += 6).
    - If enemy passes the screen bottom (enemyY > 700), reset enemy to Y = -120, assign a random X-coordinate (160 + random (220)), increase score by +10, and play score.wav.

6. Evaluate collision logic by creating bounding boxes (playerRect and enemyRect).
7. If playerRect.intersects(enemyRect) returns true:

    - Set isGameOver = true.
    - Stop timer, halt background engine audio, and play crash.wav.

8. Repaint graphics canvas (paintComponent).

-Flowchart



-Tools and Technologies

- Programming Language: Java (JDK 8 or higher)
- GUI Libraries: Java Swing (JPanel, JFrame, Timer) & AWT (Graphics2D, Rectangle, FontMetrics)
- Audio Library: Java Sound API (javax.sound.sampled.Clip, AudioInputStream)
- Development Environment (IDE): NetBeans IDE
- Hardware: Laptop

-Experimental Design

The system is verified by executing repeated gameplay runs across different input patterns and driving scenarios. Tests verified key input responsiveness, boundary containment, enemy spawning and over taking, and collision detection accuracy. Sound effects were tested and examined during state transitions to confirm seamless looping and one-shot audio execution without latency.

4. **Implementation**

-System Architecture

The application architecture is divided into three core functional modules:

1. **Input & Control Module:** Uses Java KeyListener to capture user key presses (ENTER, LEFT, RIGHT, R).
2. **Processing & Physics Module:** Driven by javax.swing.Timer at ~60 FPS to handle coordinate updates, randomized enemy generation, timer increments, and bounding-box collision detection.
3. **Graphics & Audio Output Module:** Uses Graphics2D vector drawing inside paintComponent to render background tracks, vehicle graphics, HUD text, menu screens, and manages sound streams (Clip) using javax.sound.sampled.

-Components

- **Timer Loop Controller:** Maintains a steady ~16ms update cycle (~60 FPS).
- **Vector Renderer (drawCar):** Draws player and enemy vehicles dynamically using custom vector shapes (fillRoundRect, fillOval, fillRect).
- **Bounding Box Physics Engine:** Evaluates spatial intersections using java.awt.Rectangle.
- **State Machine Controller:** Switches application states between Welcome Screen, Active Gameplay, and Game Over Screen using boolean flags.
- **Classpath Sound Engine:** Loads uncompressed .wav audio clips from classpath resources (/weeks/) using relative resource URLs.

-Audio Asset System

Managed through classpath resource paths (/weeks/filename.wav) to prevent file loading issues across operating systems:

- **welcome.wav:** Continuous loop on welcome screen.
- **run.wav:** Continuous loop during active driving.
- **score.wav:** One-shot clip played when successfully dodging an enemy car.
- **crash.wav:** One-shot clip played on vehicle impact.

These audio systems are implemented to make the game much interactive and interesting. That helps us to understand the better approach of the sound implementation in Java games and makes a foundation of GUI based games using Java. y incorporating non-blocking, multi-channel sound clips via the Java Sound API, the application delivers immediate audio feedback for player actions and collision events without interrupting the primary frame rendering pipeline. Furthermore, utilizing classpath resource loaders ensures that background music loops and event-driven sound effects operate smoothly across different operating systems.

-Weekly Workflow

Development Phase	Key Implementation Focus	Core Java Features Used
Weeks 2–4	Window Creation & Base Track GUI	JFrame, JPanel, Graphics, Color
Weeks 5–6	Object Rendering & Movement	Graphics2D, fillRoundRect( ), drawOval( ), Basic Loops
Week 7	Game Loop & Animation	javax.swing.Timer, ActionListener, Object Offset Logic
Week 8	Physics, Controls & Audio Integration	KeyListener, java.awt.Rectangle.intersects( ), javax.sound.sampled.Clip
Week 9	States (Welcome/GameOver), UI & Restart	Boolean States (isWelcomeScreen, isGameOver), Font Metrics, Resource Reset

5. **Results & Analysis**

-Result

The application successfully executes a 2D racing environment with responsive keyboard controls, dynamic scrolling tracks, and immediate audio feedback. Testing confirmed that vehicle controls respond smoothly within road constraints (150 < playerX < 390). Enemy vehicles spawn at random horizontal positions upon passing off-screen, incrementing the score by +10 points per car spawn and triggering required point audio file. Collision tests confirmed that vehicle impacts halt the game loop instantaneously, trigger crash sound effects, and display a Game Over overlay showing final score, survived time, and a restart prompt.

-Analysis

The results confirm the reliability of Swing's event-driven architecture when executing real-time 2D games. Utilizing javax.swing.Timer ensures frame rendering remains synchronized

with state updates without requiring complex manual thread management. Bounding-box detection via Rectangle.intersects() provides pixel-accurate collision verification at 60 FPS without introducing performance degradation. Furthermore, using classpath resource streams (getClass().getResource()) guarantees cross-platform audio execution without broken file paths.

-Code (Output)

Here is the output picture of the code implemented in week 9. The final outputs of the GUI based car racing game with all features inserted. The primary interface displays an attractive Welcome Screen complete with title banners, control key guides, and a preview of the player car rendered in cyan and yellow graphics. Upon pressing the start key, the application transitions smoothly into active gameplay, rendering a central dark gray road flanked by green grass margins and white boundary lines. Animated yellow lane dividers scroll continuously down the screen to simulate high-speed movement, while a heads-up display (HUD) in the upper-left corner tracks the player's live score and total survival time in seconds. The player vehicle and dynamically spawned enemy vehicles navigate the track using custom vector shapes, complete with windshields, wheels, headlights, and tail lights. Upon a vehicle impact, a semi-transparent dark overlay covers the viewport to clearly display the Game Over state alongside the player's final score, total survived time, and an instant restart prompt. The visual output is given below:



Figure1: The welcome screen of the game

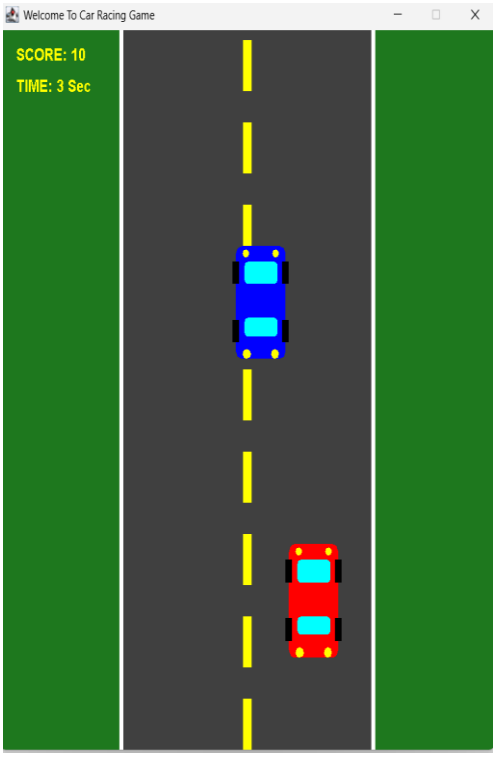


Figure2: The car passing Enemy Car

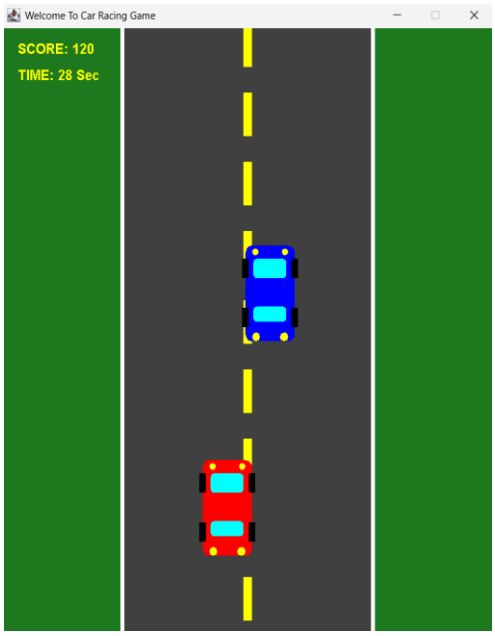


Figure3: The score and time update



Figure4: The game over with restart

6. **Discussion**

-Interpretation

The implementation results demonstrate that Java Swing and AWT are fully capable of handling 2D game loops, vector graphics rendering, state management, and real-time audio synchronization. Isolating logic into three finite states (isWelcomeScreen, active gameplay, isGameOver) simplifies event handling and prevents input processing during incorrect game phases. The system successfully meets all target goals: rendering responsive visuals, enforcing physical constraints, tracking player statistics, and maintaining non-blocking audio playbacks.

-Key Technical Achievements

- **State Machine Architecture:** Implementing distinct boolean flags (isWelcomeScreen and isGameOver) made it straightforward to isolate screen rendering, sound triggers, and perfect input processing. This design prevents from control inputs or audio clips from firing at incorrect times and ensure perfect time implementation.
- **Resource Management & Sound:** Transitioning to classpath resource paths (/weeks/filename.wav) resolved file path resolution issues across different IDE environments and operating systems. Furthermore, using separate Clip objects allowed smooth background looping alongside one-shot sound effects without causing audio latency.
- **Real-time Collision Logic:** Utilizing AWT's Rectangle.intersects() provided a lightweight, reliable bounding-box method for real-time collision detection at 60 FPS. By continuously comparing the dynamic boundary dimensions of the player's vehicle against incoming enemy cars during each timer tick, the system evaluates overlaps

instantaneously without impacting game performance. This ensures immediate state transitions to the Game Over screen perfectly.

-Challenges

- **Audio Resource Pathing:** Preventing file path errors when packaging audio assets was solved by accessing resource streams via getClass().getResource().
- **Frame-Rate Synchronization:** Ensuring road scrolling and timer increments remain smooth at ~60 FPS was achieved using fixed-delay javax.swing.Timer execution.
- **Smooth State Transitions:** Halting and starting background sound loops during state transitions required careful clip lifecycle management (stop() and loop()).

-Limitations & Potential Enhancements

While the project meets all functional requirements, future iterations could expand the gameplay experience:

1. Dynamic Difficulty Through Creating Levels: Increasing vehicle speed or narrowing the road as time progresses by implementing various level option to make the game more challenging over time.
2. Multiple Obstacles: Spawning multiple enemy cars simultaneously across different lanes.
3. High Score Persistence: Saving top scores locally to a file or database so player achievements persist after closing the application.

7. Conclusion

-Summary

The Java Car Racing Game project successfully demonstrates how foundational Java concepts—such as event handling, multi-threading via Swing timers, vector graphics, and audio processing—can be combined to build an engaging 2D game. By incrementally building the project from track rendering (Week 2) to sound integration (Week 8) and state-based game management (Week 9), the software architecture remains clean, maintainable, and robust. The resulting application provides an attractive welcome interface, smooth controls, reactive sound effects, and a seamless restart mechanism, meeting all target objectives.

-Contributions

The project demonstrates how native Java libraries can build an interactive arcade game without external game engines. Key contributions include establishing a 60 FPS Swing timer loop, building an efficient vector vehicle renderer, managing bounding-box collision physics, implementing a finite state machine, and embedding cross-platform audio handling. This project serves as an extensible foundation for learning Java GUI and 2D game development.